ME-2 Digital Twin Baseline Construction

Week 6: Dynamic Response Study

Blayne Montaque

Spring 2026

1 Purpose

The purpose of this stage is to evaluate the dynamic vibration response of the instrumented structural testbed using experimentally collected sensor data. Unlike the previous static baseline study, this stage focuses on time-varying behavior produced by vibration excitation and structural response.

The analysis uses experimentally recorded measurements stored in the file:

`data.csv`

and processed using the MATLAB implementation:

`dynamic_model.m`

The goal of this study is to characterize how the structure responds dynamically under vibration excitation by analyzing both the time-domain response and the frequency-domain behavior of the measured acceleration signals.

This stage expands the behavioral digital twin from a purely static baseline into a dynamic response framework capable of identifying vibration characteristics, response magnitude, and dominant excitation frequencies.

2 Background

Earlier stages of the project established:

- Digital twin scope and modeled variables
- Sensor calibration procedures

- Data quality review methods
- Expected-response relationships
- Static steady-state baseline behavior

The current stage introduces dynamic structural behavior through vibration excitation. The structure is subjected to controlled vibration using a motor excitation source while the load cells and accelerometer capture the resulting response.

Dynamic behavior is important because many structural systems experience time-varying loading rather than purely static conditions. A digital twin intended for structural monitoring must therefore be capable of interpreting vibration behavior in addition to steady-state response.

3 Experimental Data Source

The experimental measurements are stored in:

`data.csv`

The MATLAB script assumes the CSV file contains six numerical columns with no column headers.

Table 1: CSV Data Column Structure

Column	**Variable**	**Description**
Column 1	Load Cell 1	Primary load cell measurement
Column 2	Load Cell 2	Secondary load cell measurement
Column 3	Vibration Magnitude	Vibration motor excitation magnitude
Column 4	Accelerometer X	Acceleration in the x direction
Column 5	Accelerometer Y	Acceleration in the y direction
Column 6	Accelerometer Z	Acceleration in the z direction

The script assumes that each row corresponds to one synchronized measurement instant from the experimental system.

The sampling interval used in the implementation is:

$$\Delta t = 0.01 \text{ s}$$

which corresponds to:

$$100 \text{ Hz}$$

sampling frequency.

4 MATLAB Implementation Overview

The dynamic response analysis is implemented in:

`dynamic_model.m`

The MATLAB script performs the following tasks:

1. Reads the experimental CSV dataset
2. Extracts load cell, vibration, and accelerometer signals
3. Removes invalid rows containing missing values
4. Constructs a physical time vector
5. Computes resultant acceleration magnitude
6. Computes time-domain response metrics
7. Performs FFT-based frequency-domain analysis
8. Identifies the dominant vibration frequency
9. Generates visualization plots

5 Robust CSV Import Method

The MATLAB implementation uses a robust CSV-reading procedure to account for multiple formatting possibilities within the dataset.

The script first attempts to read the file using:

```
readcell(fileName)
```

Two possible cases are then handled.

5.1 Case 1: Comma-Separated Text in One Column

If MATLAB reads the entire row as a single text string, the script parses the numeric values manually using:

`sscanf`

This ensures that the dataset can still be processed correctly even if the CSV formatting is inconsistent.

5.2 Case 2: Properly Parsed Numeric Columns

If MATLAB correctly separates the CSV into numerical columns automatically, the script converts the data directly into matrix form using:

`cell2mat`

This makes the implementation more robust to formatting variations in exported experimental datasets.

6 Signal Extraction

After importing the dataset, each column is extracted into a dedicated MATLAB variable.

Table 2: MATLAB Signal Variables

MATLAB Variable	**Signal**	**Purpose**
`LC1`	Load Cell 1	Measures structural loading response
`LC2`	Load Cell 2	Secondary structural loading response
`Vibe`	Vibration Magnitude	Represents motor excitation level
`Xacc`	Accelerometer X	Dynamic response in the x direction
`Yacc`	Accelerometer Y	Dynamic response in the y direction
`Zacc`	Accelerometer Z	Dynamic response in the z direction

Rows containing invalid or missing data are removed using logical indexing so that all remaining signals remain synchronized.

7 Time Vector Construction

The script converts the sample index into a physical time scale using:

$$t_i = i\Delta t$$

where:

- t_i is the physical time,
- i is the sample index,
- Δt is the sampling interval.

Using a physical time scale instead of sample number allows the vibration response to be interpreted more realistically.

8 Resultant Acceleration Magnitude

The accelerometer records motion along three orthogonal axes. To describe the total vibration response of the structure, the script computes the resultant acceleration magnitude:

$$a_{\text{mag}} = \sqrt{a_x^2 + a_y^2 + a_z^2}$$

This produces a single combined acceleration signal representing the overall dynamic response magnitude independent of sensor orientation.

The resultant acceleration signal becomes the primary signal used for both time-domain and frequency-domain analysis.

9 Time-Domain Metrics

Several statistical quantities are computed from the resultant acceleration signal.

Table 3: Time-Domain Response Metrics

Metric	**MATLAB Variable**	**Purpose**
Mean Acceleration	`aMean`	Average vibration magnitude
RMS Acceleration	`aRMS`	Effective vibration energy level
Standard Deviation	`aStd`	Measures variation in response
Maximum Acceleration	`aMax`	Peak observed response magnitude
Tolerance Band	`tol`	Defines acceptable response variation

The RMS acceleration is computed as:

$$a_{\text{RMS}} = \sqrt{\frac{1}{N}\sum_{i=1}^{N} a_i^2}$$

RMS acceleration is important because it provides a measure of overall vibration intensity rather than only instantaneous peaks.

10 Tolerance Envelope

The script defines a statistical tolerance band around the mean response using:

$$\text{tol} = k\sigma$$

where:

- k is a selected multiplier,
- σ is the standard deviation of the resultant acceleration signal.

The implementation currently uses:

$$k = 2$$

The tolerance envelope defines the expected range of normal vibration response within the baseline dataset.

11 Frequency-Domain Analysis

The MATLAB implementation performs frequency-domain analysis using the Fast Fourier Transform (FFT).

Before computing the FFT, the mean value is removed from the signal:

$$a_{\text{detrended}} = a_{\text{mag}} - \bar{a}$$

This removes the DC component and isolates the oscillatory vibration behavior.

The FFT is then computed using:

$$Y = \texttt{fft}(a_{\text{detrended}})$$

The resulting spectrum is converted into a single-sided FFT magnitude spectrum for interpretation.

12 Dominant Frequency Identification

The dominant frequency is identified as the frequency corresponding to the maximum FFT magnitude.

$$f_{\text{dom}} = f(\text{argmax}(P_1))$$

where:

- P_1 is the single-sided FFT magnitude spectrum,
- f is the frequency vector.

The dominant frequency represents the strongest vibration response component present in the measured data.

This value is important because it provides information about the dynamic characteristics of the structure and the excitation behavior introduced by the vibration motor.

13 Generated Visualizations

The MATLAB implementation generates a multi-panel visualization of the experimental response.

Table 4: Generated Dynamic Response Plots

Plot	**Purpose**
Vibration Magnitude vs Time	Visualizes excitation input behavior
Load Cell Responses vs Time	Shows dynamic loading response of the structure
Accelerometer Components vs Time	Shows vibration response in each axis direction
FFT Magnitude Spectrum	Shows dominant frequency content of the resultant acceleration

These plots allow both qualitative and quantitative evaluation of the structural vibration behavior.

14 Connection to the Digital Twin

The dynamic response study extends the behavioral digital twin beyond static relationships by incorporating time-varying structural behavior.

The resulting metrics contribute to:

- Dynamic response characterization
- Baseline vibration signatures
- Frequency-domain behavioral mapping
- Expected vibration-response envelopes
- Future anomaly detection workflows

The dynamic response model allows the digital twin to characterize not only the magnitude of the response, but also how the response evolves over time and frequency.

15 Limitations

Several simplifying assumptions are present in the current implementation.

- The structure is assumed to remain within the linear operating range.
- Damping is not explicitly modeled.

- The FFT analysis assumes approximately stationary vibration behavior.
- Temperature effects are not yet incorporated into the dynamic model.
- Only one sampling interval is considered.

These simplifications are acceptable for the current baseline development stage.

16 Summary

Week 6 introduces the first dynamic response characterization stage of the behavioral digital twin. Using experimentally collected sensor data from `data.csv`, the MATLAB implementation analyzes both time-domain and frequency-domain structural behavior.

The study computes vibration-response metrics, identifies dominant frequency behavior, and generates baseline dynamic response characteristics that can later be used for anomaly detection, condition monitoring, and AI-assisted behavioral analysis.

A Appendix A: MATLAB Implementation

The complete MATLAB implementation used for the dynamic response study is provided below. The script reads the experimental file `data.csv`, extracts the load cell, vibration, and accelerometer columns, computes dynamic response metrics, performs FFT analysis, and generates the required plots.

```
%
    ------------------------------------------------------------------------

% Experimental Sensor Data Visualization and Frequency-Domain
    Analysis
%
% This script:
% 1. Reads experimental sensor data from a CSV file
% 2. Extracts load cell, vibration magnitude, and accelerometer
    signals
% 3. Computes the resultant acceleration magnitude from the X, Y,
    and Z
%    accelerometer components
% 4. Computes key time-domain signal metrics, including mean, RMS,
%    standard deviation, maximum absolute acceleration, and
    tolerance band
% 5. Computes the Fast Fourier Transform (FFT) of the resultant
%    acceleration signal
% 6. Identifies the dominant response frequency and corresponding
    FFT
%    magnitude
% 7. Plots:
%    - Vibration motor magnitude vs time
%    - Load Cell 1 and Load Cell 2 responses vs time
%    - X, Y, and Z accelerometer components vs time
%    - FFT magnitude vs frequency for the resultant acceleration
    signal
%
% Assumptions:
%   - The input file is a CSV file with NO column headers
%   - Column 1 = Load Cell 1
%   - Column 2 = Load Cell 2
%   - Column 3 = Vibration Magnitude
%   - Column 4 = Accelerometer X
%   - Column 5 = Accelerometer Y
```

```
%    - Column 6 = Accelerometer Z
%    - Each data point is collected every 10 ms
%
    ------------------------------------------------------------------------


clc; clear; close all;

kTol = 2;

%% User settings
fileName = 'data.csv';
dt = 0.01;                          % 10 ms sampling interval [s]
Fs = 1/dt;                          % sampling frequency [Hz]

%% Robust CSV read
raw = readcell(fileName);

if size(raw,2) == 1
    % Case: data stored as comma-separated text in one column
    data = cellfun(@(x) sscanf(x, '%f,%f,%f,%f,%f,%f')', ...
                   raw, 'UniformOutput', false);
    data = vertcat(data{:});
else
    % Case: CSV already parsed into separate numeric columns
    data = cell2mat(raw);
end

%% Extract columns
LC1  = data(:,1);
LC2  = data(:,2);
Vibe = data(:,3);
Xacc = data(:,4);
Yacc = data(:,5);
Zacc = data(:,6);

% Remove invalid rows
validIdx = ~(isnan(LC1) | isnan(LC2) | isnan(Vibe) | ...
             isnan(Xacc) | isnan(Yacc) | isnan(Zacc));

LC1  = LC1(validIdx);
```

```
LC2  = LC2(validIdx);
Vibe = Vibe(validIdx);
Xacc = Xacc(validIdx);
Yacc = Yacc(validIdx);
Zacc = Zacc(validIdx);

LC1  = LC1(:);
LC2  = LC2(:);
Vibe = Vibe(:);
Xacc = Xacc(:);
Yacc = Yacc(:);
Zacc = Zacc(:);

%% Time vector
N = length(Xacc);
t = (0:N-1)' * dt;

%% Resultant acceleration magnitude
AccelMag = sqrt(Xacc.^2 + Yacc.^2 + Zacc.^2);

%% Time-domain metrics
aMean = mean(AccelMag);
aRMS  = rms(AccelMag);
aStd  = std(AccelMag);
aMax  = max(abs(AccelMag));
tol   = kTol * aStd;

upperBand = aMean + tol;
lowerBand = aMean - tol;

%% Frequency-domain analysis
aDetrend = AccelMag - mean(AccelMag);

Yfft = fft(aDetrend);
P2 = abs(Yfft/N);
P1 = P2(1:floor(N/2)+1);
P1(2:end-1) = 2*P1(2:end-1);

f = Fs*(0:floor(N/2))/N;

[peakMag, peakIdx] = max(P1);
```

```
domFreq = f(peakIdx);

%% Print results
fprintf('\n');
fprintf('Experimental Dynamic Response Results\n\n');

fprintf('Time-domain metrics (resultant acceleration magnitude):\n')
    ;
fprintf('    Mean Acceleration   = %.6f g\n', aMean);
fprintf('    RMS Acceleration    = %.6f g\n', aRMS);
fprintf('    Std Acceleration    = %.6f g\n', aStd);
fprintf('    Max |Acceleration|  = %.6f g\n', aMax);
fprintf('    Tolerance Band      = +/- %.6f g\n\n', tol);

fprintf('Frequency-domain metrics:\n');
fprintf('    Dominant Frequency  = %.6f Hz\n', domFreq);
fprintf('    Peak FFT Magnitude  = %.6f\n\n', peakMag);

%% Create centered figure window
figure('Units','normalized', ...
       'Position',[0.06, 0.12, 0.88, 0.72], ...
       'Color','w', ...
       'Name','Experimental Sensor Data Overview');

tiledlayout(2,3,'TileSpacing','compact','Padding','compact');

%% Top left: Vibration motor magnitude
nexttile;
plot(t, Vibe, 'LineWidth', 1.8);
grid on;
box on;
xlabel('Time (s)', 'FontWeight','bold');
ylabel('Vibration Magnitude', 'FontWeight','bold');
title('Vibration Motor Magnitude', 'FontWeight','bold');
set(gca, 'FontSize', 11, 'LineWidth', 1.2);

%% Top middle: Load cell responses
nexttile;
plot(t, LC1, 'LineWidth', 1.8); hold on;
plot(t, LC2, 'LineWidth', 1.8);
grid on;
```

```
148 box on;
149 xlabel('Time (s)', 'FontWeight','bold');
150 ylabel('Load Cell Reading', 'FontWeight','bold');
151 title('Load Cell Responses', 'FontWeight','bold');
152 legend('Load Cell 1','Load Cell 2','Location','best');
153 set(gca, 'FontSize', 11, 'LineWidth', 1.2);
154
155 %% Top right: Accelerometer components
156 nexttile;
157 plot(t, Xacc, 'LineWidth', 1.8); hold on;
158 plot(t, Yacc, 'LineWidth', 1.8);
159 plot(t, Zacc, 'LineWidth', 1.8);
160 grid on;
161 box on;
162 xlabel('Time (s)', 'FontWeight','bold');
163 ylabel('Acceleration (g)', 'FontWeight','bold');
164 title('Accelerometer Components', 'FontWeight','bold');
165 legend('X','Y','Z','Location','best');
166 set(gca, 'FontSize', 11, 'LineWidth', 1.2);
167
168 %% Bottom row: FFT spanning all 3 columns
169 nexttile([1 3]);
170 plot(f, P1, 'LineWidth', 1.8); hold on;
171 plot(domFreq, peakMag, 'o', 'MarkerSize', 8, 'LineWidth', 1.8);
172 grid on;
173 box on;
174 xlabel('Frequency (Hz)', 'FontWeight','bold');
175 ylabel('FFT Magnitude', 'FontWeight','bold');
176 title('Frequency-Domain Response of Resultant Acceleration', '
      FontWeight','bold');
177 legend('FFT Magnitude','Dominant Frequency','Location','best');
178 set(gca, 'FontSize', 11, 'LineWidth', 1.2);
179 xlim([0 Fs/2]);
180
181 sgtitle('Experimental Sensor Data Visualization and Dynamic Response
      Analysis', ...
182         'FontWeight','bold', 'FontSize', 14);
```